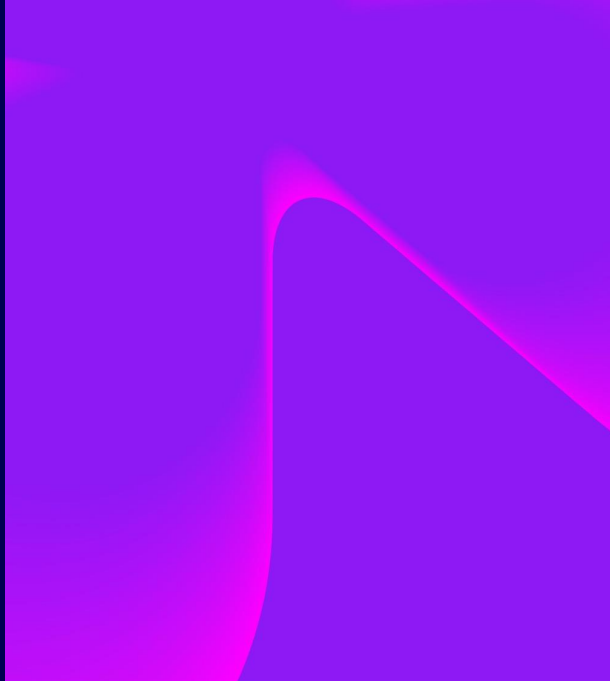
team neusta

KI-NATIVE SOFTWARE-ENTWICKLUNG

Best Practices für die KI-unterstützte Entwicklung – Impulse vom Java Forum Nord 2026

K. Renken

19. Oktober 2026



AGENDA



01 Ausgangslage

02 Impulse vom Java Forum Nord 2026

03 Best Practices

04 Team und Organisation

05 Fazit

CODING IS DEAD. SOFTWAREENTWICKLUNG LEBT MEHR ALS JE ZUVOR.

Kajetan Mischok, Java Forum Nord 2026

01

AUSGANGSLAGE

VOM AUTOCOMPLETE ZUM AGENTEN



Autocomplete	**Chat**	**Agenten**	**Orchestrierung**
Vorschläge für Zeilen und Blöcke direkt in der IDE	Fragen, Erklärungen, Snippets per Copy & Paste	planen, ändern Dateien, führen Tests aus und iterieren selbstständig	mehrere Agenten parallel, Sub-Agenten, Werkzeuge per MCP

Claude Code | **GitHub Copilot** | **Cursor** | **OpenCode** | **JetBrains AI**

TODO: Eigene Einordnung: Welche Werkzeuge nutzen wir in neusta-Projekten? Ggf. kurze Umfrage im Publikum.

ZWEI PERSPEKTIVEN AUF KI IN DER JAVA-WELT



KI für die Entwicklung

- Assistenten und Agenten im Entwicklungsalltag
- Analyse, Generierung, Tests, Review, Migration
- Auswirkungen auf Rollen, Lernen und Qualität

→ **Fokus dieses Vortrags**

KI in der Anwendung

- Spring AI, Quarkus LangChain4j
- MCP-Server in Java, RAG, Vektor-Datenbanken
- Lokale Modelle, z. B. mit Ollama

→ **nur am Rande**

Quelle: R. Fichtner, „Java 2026: So startest du (wieder) richtig durch“ (Folien); F. Hahne, „Hey Mr. TamboU(r)Ine man, create a TUI for me“ (Abstract)

02

IMPULSE VOM JAVA FORUM NORD 2026

KI AUF DEM JAVA FORUM NORD 2026



37	7	5	1
Vorträge	mit KI als Hauptthema	mit KI als Nebenaspekt	faktischer KI-Track (Hörsaal F)

- Im Mittelpunkt standen Menschen, Kompetenz und Verantwortung, nicht Werkzeuge.
- Praxisvorträge zu Agenten-Workflows, MCP oder KI-Security gab es kaum.

Quelle: Programm unter https://javaforumnord.de/2026/programm/ (Sessionize)

DIE KI-VORTRÄGE IM ÜBERBLICK

Erfahrung lässt sich nicht prompten

Nora Schöner
Opening Keynote

Kompetenz entsteht durch eigene Erfahrung und Austausch – nicht im Chat mit dem Agenten.

The Excel Moment

Gerd Aschemann
Software-Architekt

LLMs machen Architekt:innen zu Werkzeugbauern: Analyse-Tools für Legacy-Systeme in Stunden statt Wochen.

Cognitive Debt

Martin Lippert
Broadcom, Spring Tools

Wer das Denken an die KI delegiert, verliert die Fähigkeit, Code zu schreiben und zu prüfen.

Junior Devs im KI-Zeitalter

Kajetan Mischok
Mischok GmbH

TDD, CI/CD und Reviews sind der Maßstab für KI-Output. Wer nicht versteht, kann nicht verantworten.

DIE KI-VORTRÄGE IM ÜBERBLICK



Beyond Coding

Sascha Doemer
LMIS AG

Die Zukunft der Entwicklung liegt im Kontext: Problem verstehen, präzise beschreiben, Ergebnis nachweisen.

Developing Software Fast and Slow

Oliver Zeigermann
Closing Keynote

Vibe Coding für Wegwerf-Prototypen, Produktivcode „slow“ mit guter Architektur.

„Wir werden von der KI …“

Jens Schauder
Spring Data

Bewusst pessimistischer Blick auf die Folgen für Wirtschaft, Gesellschaft und Demokratie.

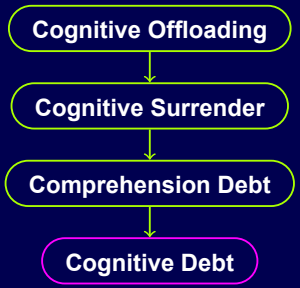
Am Rande

Spring AI & Quarkus LangChain4j
R. Fichtner

Java-TUI mit LangChain4j
F. Hahne

KI-Anti-Patterns im Code Review
M. Landreh, L. Eisbach

COGNITIVE DEBT: WAS KI MIT UNSEREN KÖPFEN MACHT



Studie: neue Bibliothek lernen

50 % Quiz mit KI

65,5 % Quiz ohne KI

Kaum schneller: 23 statt 24,7 Minuten

Entscheidend ist, *wie* man KI nutzt

× AI Delegation, Progressive AI Reliance, Iterative AI Debugging

✓ Generation-Then-Comprehension, Hybrid Code-Explanation, Conceptual Inquiry

TODO: Zahlen vor dem Vortrag mit der Originalstudie abgleichen.

Quelle: M. Lippert, „Cognitive Debt" (Folien, Speaker Deck), zitiert Anthropic, „How AI Impacts Skill Formation", und Addy Osmani („Cognitive Surrender")

DEVELOPING SOFTWARE FAST AND SLOW



Fast – System 1

- Mensch: schnell, intuitiv, erfahrungsbasiert
- LLM: direkte Fortführung des Kontexts
- Vibe Coding als PoC: herausfinden, was die Leute wirklich wollen

Slow – System 2

- Mensch: langsam, analytisch
- LLM: Reasoning, agentische Loop, Tool-Calls
- Produktivcode mit guter Architektur und den Erkenntnissen aus dem PoC neu bauen

„AI kann für nichts Produktives die Verantwortung übernehmen.“

Deshalb: PoC nach Abschluss in den Müll.

Quelle: O. Zeigermann, „Developing Software Fast and Slow“, Closing Keynote (Folien)

DIE ZUKUNFT LIEGT IM KONTEXT



> Die Zukunft der Entwicklung gehört nicht denen, die nur schneller Code produzieren. Sie gehört denen, die bessere Fragen stellen, bessere Kontexte liefern, bessere Entscheidungen treffen – und beweisen können, dass das Ergebnis wirklich funktioniert.
>
> – Sascha Doemer, „Beyond Coding“ (Abstract)

DER GEMEINSAME NENNER



✓ Verantwortung bleibt beim Menschen	Mischok, Zeigermann
✓ Kompetenz entsteht durch eigenes Denken und Austausch	Schöner, Lippert
✓ Kontext und Spezifikation werden zur Kernkompetenz	Doemer
✓ Engineering-Grundlagen sind der Maßstab	Mischok, Landreh/Eisbach
✓ Prototyp und Produkt sauber trennen	Zeigermann
✓ LLMs als Werkzeugbauer nutzen	Aschemann

Lücke: Konkrete Agenten-Praxis, Security und Governance → ergänzen wir im nächsten Teil.

03

BEST PRACTICES

ACHT PRAKTIKEN IM ÜBERBLICK



01
Kontext ist das neue Coding

02
Tests und Pipeline als Leitplanken

03
Kleine Schritte, echte Reviews

04
Fast für Prototypen, slow für Produkte

05
Denken nicht delegieren

06
KI als Werkzeugbauer

07
Agenten gezielt einsetzen

08
Sicher und souverän arbeiten

Gefüllt: abgeleitet aus den JFN-Vorträgen Umrandet: ergänzt um aktuelle Praxis

01 · KONTEXT IST DAS NEUE CODING

Warum?

Agenten kennen weder Fachlichkeit noch Architektur noch Teamkonventionen. LLMs tun sich schwer zu erkennen, was wichtig ist und wann sie nachfragen sollten.

Konkret

- Projektwissen versionieren: `AGENTS.md` / `CLAUDE.md` mit Build-Befehlen, Konventionen und No-Gos
- Architekturentscheidungen (ADRs) und Fachbegriffe auffindbar machen
- Erst Plan und Spezifikation mit Akzeptanzkriterien, dann Umsetzung
- Kontext kuratieren statt fluten

TODO: Beispiel einer AGENTS.md aus einem neusta-Projekt zeigen.

Quelle: S. Doemer (Abstract), O. Zeigermann (Folien); konkrete Praktiken ergänzt

02 · TESTS UND PIPELINE ALS LEITPLANKEN



Warum?

TDD, CI/CD und Code Reviews sind „Maßstab für das, was die KI liefert“. Ohne automatisches Feedback skaliert nur die Menge, nicht die Qualität.

Konkret

- Test zuerst – selbst geschrieben oder bewusst geprüft
- Agent führt Build, Tests und Linter selbst aus
- Architekturregeln automatisieren, z. B. mit ArchUnit
- KI-generierte Tests prüfen: Testen sie Verhalten oder nur die Implementierung?

Quelle: K. Mischok (Abstract), M. Landreh/L. Eisbach (Handout); ArchUnit ergänzt

03 · KLEINE SCHRITTE, ECHTE REVIEWS

Anti-Pattern	Gegenmittel
× Kontrollverlust über generierten Code	✓ Assistiertes Coding: der Mensch steuert
× Blindes Vertrauen in KI-generierte Tests	✓ Tests überprüfen
× Masse statt Klasse	✓ Kleine Pull Requests
× Lastverschiebung statt Liefereffizienz	✓ Fokus auf Anforderungen

Wer committet, verantwortet – auch generierten Code.

Quelle: M. Landreh, L. Eisbach, „Code Reviews – Wie man es so richtig falsch macht“ (Handout); Zuordnung sinngemäß

04 · FAST FÜR PROTOTYPEN, SLOW FÜR PRODUKTE



Fast: Vibe Coding

- Anforderungen klären, Ideen in Stunden testen
- Feedback von Nutzer:innen einholen
- Code bewusst als Wegwerfprodukt kennzeichnen

Slow: Produktentwicklung

- Architektur, Spezifikation, Tests, Review
- KI als Werkzeug, der Mensch entscheidet
- Neu bauen statt Prototyp „hochschieben“

TODO: Beispiel: PoC aus einem Kundenprojekt, der bewusst neu gebaut wurde.

Quelle: O. Zeigermann, „Developing Software Fast and Slow“ (Folien)

05 · DENKEN NICHT DELEGIEREN

Fördert Verständnis ✓

- Erst selbst lösen oder skizzieren, dann mit der KI vergleichen
- Generierten Code erklären lassen und nachvollziehen
- Konzeptfragen statt fertiger Lösungen

Baut Cognitive Debt auf ×

- Komplette Aufgaben delegieren
- Schleichend wachsende Abhängigkeit
- Fehler nur per KI „wegdebuggen“

Erfahrung entsteht durch Tun – und vertieft sich im Austausch: Pairing, Reviews, Community.

Quelle: M. Lippert, „Cognitive Debt“ (Folien); N. Schöner, „Erfahrung lässt sich nicht prompten“ (Abstract)

06 · KI ALS WERKZEUGBAUER



Idee

- Static Analyzer beantworten nicht, *warum* ein System so gebaut ist
- Antworten stecken in Tickets, Wikis, Mailinglisten und Build-Skripten
- Der Coding-Assistent baut die fehlenden Analyse-„Sensoren“

Anwendung

- Eine abfragbare Sicht auf Code, Build-Metadaten, Tickets und Fachdoku
- Abgleich mit der Ubiquitous Language
- Migrationen vorbereiten, z. B. mit OpenRewrite-Rezepten

TODO: Legacy-Beispiel aus einem neusta-Projekt ergänzen.

Quelle: G. Aschemann, „The Excel Moment“ (Abstract); OpenRewrite nach R. Fichtner (Folien)

07 · AGENTEN GEZIELT EINSETZEN



- Gut abgegrenzte Aufgaben mit klarem Done-Kriterium
- Fester Zyklus: **Erkunden** **Planen** **Umsetzen** **Verifizieren** **Committen**
- Berechtigungen bewusst setzen: Sandbox, Freigaben für Shell und Netzwerk, keine Produktivzugänge
- Werkzeuge über MCP anbinden, z. B. Ticketsystem, Doku, Datenbanken (lesend)
- Parallelisieren in getrennten Worktrees und Ergebnisse wie fremde Pull Requests behandeln

TODO: Eigene Erfahrungen und Werkzeug-Setup ergänzen.

08 · SICHER UND SOUVERÄN ARBEITEN



- **Daten:** keine Secrets und Kundendaten in Prompts, nur freigegebene Tools und Modelle
- **Prompt Injection:** Tickets, Webseiten und Abhängigkeiten sind nicht vertrauenswürdige Eingaben
- **Supply Chain:** halluzinierte Paketnamen und Lizenzen prüfen

TODO: Interne KI-Richtlinie von team neusta verlinken.

- **Nachvollziehbarkeit:** KI-Beteiligung in Commits und Pull Requests kennzeichnen
- **Regeln:** EU AI Act, interne KI-Richtlinie, Kundenverträge
- **Souveränität:** Abhängigkeit von Anbietern bewusst steuern

LIVE-DEMO: VOM TICKET ZUM PULL REQUEST



TODO: Demo-Szenario festlegen (Java/Spring-Projekt), Backup-Video aufnehmen.

04

TEAM UND ORGANISATION

ROLLEN VERSCHIEBEN SICH

Problem verstehen **Lösungsraum formen** **Umsetzung kritisch prüfen**

Ergebnis absichern

Kernkompetenzen

Architekturverständnis **Präzise Requirements** **Kontextmodellierung**

Fachliches Review **Verifikation** **Kommunikation**

Kritisches Denken **Übersetzen zwischen Fachlichkeit, Technik und Betrieb**

Quelle: S. Doemer, „Beyond Coding“ (Abstract)

JUNIOR DEVS AUSBILDEN



- Verstehen vor Übernehmen: Was ich nicht verstehe, kann ich nicht verantworten
- Engineering-Grundlagen bewusst trainieren: TDD, CI/CD, Code Reviews
- Lernmodus statt Delegationsmodus fördern
- Tugenden, die keine KI übernimmt: Selbstorganisation, Verantwortung, im Team wachsen
- Erfahrung teilen: Pairing, Reviews, Community

Offene Frage: Wie entsteht ein gutes „System 1“, wenn die KI das Üben übernimmt?

Quelle: K. Mischok (Abstract), M. Lippert (Folien), N. Schöner (Abstract), O. Zeigermann (Folien)

DER KRITISCHE BLICK

Risiken

- Folgen für Wirtschaft, Gesellschaft und Demokratie
- KI als „großer Gleichmacher“: Mainstream, kein Geschmack
- Cognitive Surrender: Vertrauen auch in falsche Antworten

TODO: Eigene Haltung / neusta-Position formulieren.

Unsere Antwort

- KI bewusst und begründet einsetzen
- Wirkung an Qualität und Lieferfähigkeit messen, nicht an Codezeilen
- Abhängigkeiten von Anbietern steuern

Quelle: J. Schauder (Abstract), O. Zeigermann (Folien), M. Lippert (Folien)

05

FAZIT

TAKEAWAYS

1. Verantwortung bleibt beim Menschen – für jeden Commit.
2. Kontext und Spezifikation sind die neuen Kernkompetenzen.
3. Tests, Pipeline und Reviews sind der Maßstab für KI-Output.
4. Fast für Prototypen, slow für Produkte.
5. Denken nicht delegieren – Erfahrung aufbauen und teilen.

ERFAHRUNG LÄSST SICH NICHT PROMPTEN.

Nora Schöner, Opening Keynote, Java Forum Nord 2026

team neusta

VIELEN DANK!

Fragen und Diskussion

K. Renken · k.renken@neusta.de

QUELLEN



- Programm: https://javaforumnord.de/2026/programm/
- N. Schöner: Erfahrung lässt sich nicht prompten (Opening Keynote)
- G. Aschemann: The Excel Moment: AI for Legacy Architecture Analysis
- M. Lippert: Cognitive Debt – Was KI mit unseren Köpfen macht
  Folien: https://speakerdeck.com/martinlippert/cognitive-debt-what-ai-will-do-to-us
- K. Mischok: Ausbildung von Junior Devs im KI-Zeitalter
- J. Schauder: Wir werden von der KI … (Titel gekürzt)
- S. Doemer: Beyond Coding: Warum die Zukunft der Entwicklung im Kontext liegt
- O. Zeigermann: Developing Software Fast and Slow (Closing Keynote) – Folien
- M. Landreh, L. Eisbach: Code Reviews – Wie man es so richtig falsch macht – Handout
- R. Fichtner: Java 2026: So startest du (wieder) richtig durch – Folien
- F. Hahne: Hey Mr. TamboU(r)Ine man, create a TUI for me